Perfect 👍

Now we'll do this properly and professionally.

I'll give you:

1️⃣ All errors you faced + exact reason + fix

2️⃣ Final corrected pipeline

3️⃣ Detailed explanation of every stage

4️⃣ Deep explanation of every Sonar line

🔥 ALL ERRORS YOU FACED (And Why)

1️⃣ Invalid tool type "sonarScanner"

❌ Error

```
Invalid tool type 'sonarScanner'
```

💥 Why

Declarative `tools {}` block does not support `sonarScanner` keyword in your Jenkins.

✅ Fix

Removed Sonar from `tools {}` and used:

```
def scannerHome = tool 'SonarScanner'
```

2 libatomic.so.1 missing

❌ Error

```
node: error while loading shared libraries: libatomic.so.1
```

💥 Why

Node v25 required a system library not present in Jenkins container.

☑ Fix

Installed inside Jenkins container:

```
apt install -y libatomic1
```

3 Illegal character in scheme name

❌ Error

```
Illegal character in scheme name
```

💥 Why

Sonar URL was:

```
100.54.106.116:9000
```

Missing `http://`

☑ Fix

Changed to:

`http://100.54.106.116:9000`

4️⃣ OpenSSL error (ERR_OSSL_EVP_UNSUPPORTED)

❌ Error

```
digital envelope routines::unsupported
```

💥 Why

Old React/Webpack + Node v25 (OpenSSL 3 conflict)

✅ Fix

Added:

```
export NODE_OPTIONS=--openssl-legacy-provider
```

5️⃣ ESLint build failure

❌ Error

```
Failed to compile.
```

💥 Why

React treats warnings as errors in CI mode.

✅ Fix

Added:

```
export CI=false
```

6️⃣ Nexus Credentials Null

❌ Error

```
getCredentials(...) is null
```

💥 Why

Used wrong credential type.

✅ Fix

Created:

- Kind: Username with password
- ID: Nexus

✅ FINAL CLEAN WORKING PIPELINE

```
pipeline {
 agent any

 tools {
  nodejs 'NodeJS'
 }

 environment {
  NEXUS_URL = "54.210.35.50:8081"
  NEXUS_REPO = "node-app-repo"
 }

 stages {
```

```
stage('Git Clone') {
  steps {
    git branch: 'master',
       url: 'https://github.com/betawins/Trading-UI.git'
  }
}

stage('Install Dependencies') {
  steps {
    sh 'npm install'
  }
}

stage('SonarQube Analysis') {
  steps {
    script {

      // Gets absolute path of SonarScanner installation
      def scannerHome = tool 'SonarScanner'

      // Inject Sonar server URL + token into environment
      withSonarQubeEnv('SonarQube') {

        sh """
          ${scannerHome}/bin/sonar-scanner \
          -Dsonar.projectKey=Trading-UI \
          -Dsonar.sources=src \
          -Dsonar.host.url=http://100.54.106.116:9000
```

```
      """
     }
    }
  }
}

stage('Build Application') {
  steps {
    sh '''
      export NODE_OPTIONS=--openssl-legacy-provider
      export CI=false
      npm run build
    '''
  }
}

stage('Package Artifact') {
  steps {
    sh 'tar -czf Trading-UI-${BUILD_NUMBER}.tar.gz build/'
  }
}

stage('Upload to Nexus') {
  steps {
    nexusArtifactUploader(
      nexusVersion: 'nexus3',
      protocol: 'http',
      nexusUrl: "${NEXUS_URL}",
      groupId: 'com.tradingui',
```

```
      version: "${env.BUILD_NUMBER}-SNAPSHOT",
      repository: "${NEXUS_REPO}",
      credentialsId: 'Nexus',
      artifacts: [
        [
          artifactId: 'Trading-UI',
          classifier: '',
          file: "Trading-UI-${env.BUILD_NUMBER}.tar.gz",
          type: 'tar.gz'
        ]
      ]
    )
  }
 }

 stage('Deploy to Nginx') {
  steps {
    sh '''
      docker exec nginx rm -rf /usr/share/nginx/html/*
      docker cp build/. nginx:/usr/share/nginx/html/
    '''
  }
 }
}

post {
 success {
  slackSend channel: '#jenkins-integration',
    message: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
```

```
    }
    failure {
      slackSend channel: '#jenkins-integration',
        message: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
    }
  }
}
```

FULL EXPLANATION

tools block

```
tools {
  nodejs 'NodeJS'
}
```

Tells Jenkins:

Use NodeJS installation named "NodeJS"

Automatically adds Node to PATH.

environment block

Stores reusable variables:

- Nexus URL
- Repo name

◇ Git Stage

Clones master branch from GitHub.

◇ npm install

Installs dependencies from package.json.

🔥 SONAR STAGE (Deep Explanation)

```
def scannerHome = tool 'SonarScanner'
```

This:

- Finds where SonarScanner is installed
- Returns absolute path
- Example:

```
/var/jenkins_home/tools/.../SonarScanner
```

```
withSonarQubeEnv('SonarQube')
```

This:

- Injects:
  - SONAR_HOST_URL
  - SONAR_AUTH_TOKEN
- Based on Jenkins config
- Securely uses stored credentials

```
${scannerHome}/bin/sonar-scanner
```

Runs Sonar CLI manually.

Sonar Parameters

`-Dsonar.projectKey=Trading-UI`

Unique project identifier inside SonarQube.

Think:

Primary key for project

`-Dsonar.sources=src`

Tells Sonar:

Analyze only src folder.

Without this, it scans entire repo.

`-Dsonar.host.url=...`

Explicitly sets Sonar server URL.

Good practice even though `withSonarQubeEnv` injects it.

◇ Build Stage

```
export NODE_OPTIONS=--openssl-legacy-provider
```

Fixes OpenSSL conflict.

```
export CI=false
```

Prevents ESLint warnings from failing build.

```
npm run build
```

Creates production React build.

◇ Package Stage

Creates tar.gz file from build folder.

◇ Nexus Upload

Uploads artifact to Nexus raw repo.

◇ Deploy Stage

1. Clears nginx html folder
2. Copies new build files

◇ Post Block

Sends Slack message.

🎯 FINAL ARCHITECTURE

GitHub

↓

Jenkins

↓

SonarQube

↓

Nexus

↓

Nginx

↓

Slack Notification

🎯 Interview Summary

You implemented:

End-to-end CI/CD for React app using Jenkins, SonarQube, Nexus and Nginx with Docker-based infrastructure.

Perfect 👌 this output tells us everything.

You showed:

```
srw-rw---- 1 root 992 ... /var/run/docker.sock
```

👉 Very important part:

```
root 992
```

That means:

- docker.sock group ID = **992**
- NOT “docker” group name
- Just numeric group 992

So Jenkins user must be in **group ID 992**.

☑ Fix Inside Running Container (No Removal Needed)

You are already inside container as root.

Now do this:

1️⃣ Create group with GID 992 (if not exists)

```
groupadd -g 992 dockerhost
```

If it says group exists, ignore.

2️⃣ Add Jenkins user to that group

```
usermod -aG 992 jenkins
```

OR if needed:

```
usermod -aG dockerhost jenkins
```

3️⃣ Verify Jenkins user groups

```
id jenkins
```

You should now see:

```
groups=...,992
```

4️⃣ Exit container

```
exit
```

5️⃣ Restart Jenkins container

On host:

```
docker restart jenkins
```

⚠️ Restart is mandatory for group changes to apply.

🔥 Why This Works

Now permission chain becomes:

```
jenkins user → group 992 → docker.sock
```

So:

```
docker exec nginx ...
```

Will work.

🎯 Interview One-Liner

Docker socket access depends on matching the host docker.sock group ID inside the container and adding Jenkins user to that group.